Autodrive: Real-Time Autonomous Driving System Using Computer Vision and Deep Learning

Harsh Bhardwaj | Shrishti Singh | Rajat Kandpal

Department of Electronics and Communication Engineering, Maharaja Surajmal Institute of Technology, New Delhi, India.

bhardwajharsh0312@gmail.com | shrisht2723@gmail.com | rajatkandpal12@gmail.com

Abstract

Autonomous driving requires a vehicle to recognise road users and traffic infrastructure, localise itself relative to lane markings, and turn that understanding into steering commands within a tight latency budget. This paper presents Autodrive, a modular real-time driving-perception system for commodity GPU hardware that combines three deep-learning components: (i) a YOLO11s detector specialised to ten road-relevant COCO categories, (ii) a YOLO11s-seg instance-segmentation network for lane markings, trained on TuSimple annotations converted from sparse polylines into closed polygons using a width-aware centreline-offset procedure, and (iii) an end-to-end steering regressor following the NVIDIA DAVE-2 convolutional architecture. On a held-out COCO test split (525 images, 3,116 instances), the detector reaches 0.712 precision, 0.614 recall, 0.672 mAP50 and 0.471 mAP50-95. The lane network reaches 0.992 box mAP50 and 0.877-0.890 mask mAP50 on 726 validation frames, while mask mAP50-95 is 0.363-0.389, largely because IoU thresholds penalise structures only 4-12 pixels wide. Measured per-image inference times of 7.3 ms (detection) and 8.1 ms (segmentation) on a Tesla T4 project to 32-44 frames per second for the two YOLO modules. On a systematic sample of 505 steering-validation frames, the checkpoint records 22.9616 deg MAE, 33.2350 deg RMSE, +20.9977 deg mean bias, R2=-0.8769, Pearson r=-0.0404, and 9.11% within +/-5 deg. These results indicate poor steering predictions; closed-loop behavior has not been evaluated. We analyse failure modes, discuss the validity limits of offline metrics, and specify the validation required before any deployment claim.

AUTODRIVE RESEARCH TEAM | CODE & PROOF OF WORK

github.com/HA2345567/Autodrive

Keywords— autonomous driving, end-to-end learning, DAVE-2, steering prediction, YOLO11, lane segmentation, object detection, real-time perception.

I. INTRODUCTION

Background

Driving automation is commonly described by the SAE J3016 taxonomy, which separates systems that assist a human driver from those that perform the entire dynamic driving task [1]. Every level above basic assistance depends on the same perceptual primitives: knowing what is in the scene, where the drivable lane lies, and how to steer. Two design philosophies address these primitives. *Modular* pipelines decompose the problem into detection, lane or free-space estimation, planning and control, with hand-specified interfaces between stages. *End-to-end* systems learn a direct mapping from camera pixels to control commands. The end-to-end idea is old: ALVINN showed in the late 1980s that a small neural network could steer a vehicle from camera and laser input [2], and the DARPA-funded DAVE project demonstrated off-road obstacle avoidance by learning directly from human driving [3]. It returned to prominence when NVIDIA trained a nine-layer convolutional network, DAVE-2, to map a single front-camera image to a steering command and drove it on public roads [4]. Recent surveys document how much the field has since expanded, and how many open problems remain in robustness, interpretability and causal confusion [5].

In parallel, real-time object detectors descended from YOLO [6] have matured into practical, easily trained tools [7]. The Ultralytics YOLO11 family [8], [9] supports detection and instance segmentation with a unified API and models small enough for a single mid-range GPU. This makes it feasible for a small team to assemble a multi-task perception stack without industrial-scale compute.

Problem Statement

A pure end-to-end steering network is compact and fast, but it exposes no intermediate representation: when it fails, there is no explicit statement of what it believed about lanes or obstacles. A pure modular stack of large models, in contrast, can exceed the latency and memory budget of modest hardware. The problem addressed here is to build and rigorously evaluate a *lightweight* system that (a) perceives road users and traffic signals, (b) recovers lane geometry, and (c) predicts steering directly, using only publicly available data and a single commodity GPU, and to report its performance honestly, including the limits of the evaluation protocol.

Research Gap

Three gaps motivate this work. First, DAVE-2-style regressors are usually evaluated in isolation, with little explicit scene perception running alongside them to cross-check their output [4], [10]. Second, lane datasets such as TuSimple annotate lanes as sparse polylines sampled at fixed image rows [11], whereas modern instance-segmentation frameworks expect closed polygons; the conversion is rarely documented, even though it determines both the supervision signal and what any resulting mask metric means. Third, results for compact detectors are often reported on a validation split that was also used for checkpoint selection, which inflates the headline figure; held-out reporting is not always the default. Autodrive addresses each of these by pairing explicit perception with steering regression, documenting a reproducible polyline-to-polygon pipeline, and reporting held-out metrics where a held-out split exists.

Objectives

1) Fine-tune a compact YOLO11s detector on a class-balanced COCO subset covering the ten categories most relevant to road scenes, and report held-out accuracy per class.
2) Convert TuSimple polyline annotations into instance-segmentation labels and train a YOLO11s-seg lane network that runs in real time.
3) Implement a DAVE-2 convolutional regressor for steering-angle prediction following NVIDIA's published design [4].
4) Characterise the combined latency budget and identify the evaluation limits that must be closed before any on-road claim.

Contributions

- **System:** a three-branch perception-to-steering architecture (Fig. 1) that pairs explicit detection and lane segmentation with an end-to-end steering regressor.
- **Data engineering:** a width-aware centreline-to-polygon conversion for TuSimple lanes (Eqs. 1–5), and a round-robin, class-balanced COCO subset sampler with crowd and degenerate-box filtering.
- **Evaluation:** held-out test results for detection; a documented analysis of why mask-IoU metrics on thin lane ribbons are far lower than box metrics, with a closed-form illustration (Eq. 6).
- **Engineering:** measured per-module latencies and model footprints (≈40 MB combined weights) with a projection to combined throughput, plus an explicit validation roadmap.

The remainder of the paper is organised as follows. Section II reviews related work, Section III describes data, Section IV the models, and Sections V–VII training, experimental design and evaluation. Sections VIII–XII present results, discussion, interpretability, limitations and conclusions.

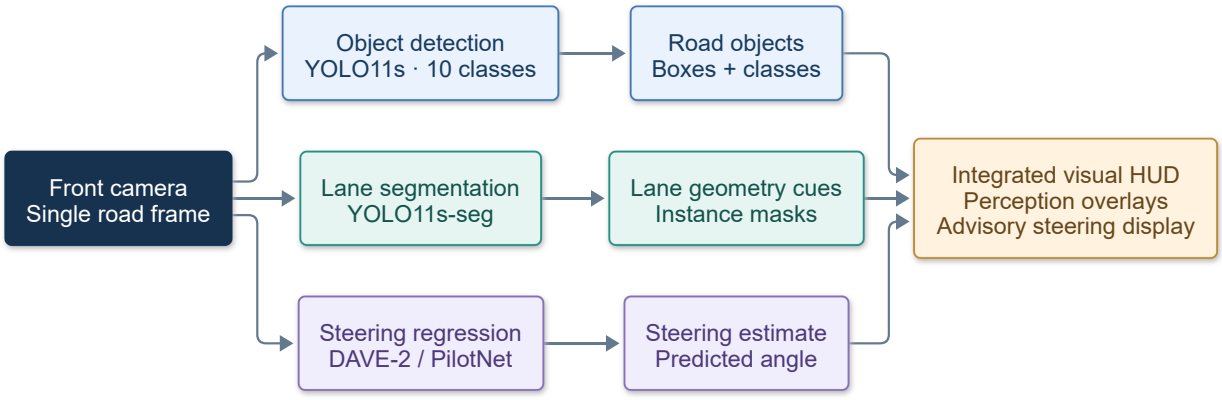


Fig. 1. Overall workflow of Autodrive. A single front-camera frame feeds three parallel branches (detection, lane segmentation, steering regression) whose outputs are combined by an integration layer.

II. LITERATURE REVIEW

Existing Techniques

End-to-end driving. ALVINN [2] and DAVE [3] established behaviour cloning from human demonstrations. DAVE-2 [4] scaled the idea with a convolutional network trained on roughly 72 hours of driving, using side cameras with synthetic shifts and rotations to teach recovery from off-centre positions. A follow-up analysis introduced VisualBackProp to show that the trained network attends to lane markings and road edges [10]. Conditional imitation learning added high-level commands so that one network can follow navigation commands at intersections [12], and large-scale video training broadened the supervision [13]. Behaviour cloning nonetheless has documented limits, including dataset bias, covariate shift and poor generalisation to dynamic scenes [14], [15].

Object detection and instance segmentation. Two-stage detectors such as Faster R-CNN [16] and Mask R-CNN [17] set accuracy baselines. Single-stage YOLO detectors [6] trade a small accuracy loss for large speed gains, and later versions have steadily improved the trade-off [7]. YOLO instance segmentation follows the prototype-mask idea of YOLACT [18]: a shared set of mask prototypes is linearly combined using per-instance coefficients. YOLO11 [8], [9] is anchor-free, uses decoupled heads and refined backbone blocks, and its box regression combines Complete-IoU [19] and distribution focal loss [20] terms.

Lane detection. Lane detection has been approached as semantic segmentation, instance segmentation and row-wise classification. SCNN passes messages across rows and columns to capture the long, thin structure of lanes [21]. LaneNet casts the problem as binary segmentation plus instance embedding [22]. Self-attention distillation produces lightweight networks [23], and row-wise formulations reach very high frame rates [24]. LaneATT uses anchor-based attention for real-time operation [25]. The TuSimple benchmark [11] evaluates point-wise accuracy and false-positive/false-negative rates at fixed row positions, not mask IoU.

Comparative Review

Table I. Representative approaches and their position relative to Autodrive

Approach	Task	Typical data	Output	Limitation relevant here
DAVE-2 [4]	Steering	Own 72 h road data	Steering command	No explicit scene output; data not public
Cond. imitation [12]	Driving policy	CARLA [26]	Controls given command	Simulator-bound evaluation
Faster / Mask R-CNN [16], [17]	Detection / instance seg.	COCO [27]	Boxes / masks	Heavier, slower than YOLO
SCNN, LaneNet [21], [22]	Lane segmentation	TuSimple, CULane	Lane masks / instances	Lane-only; no steering link
UFLD, LaneATT [24], [25]	Lane detection	TuSimple, CULane	Lane polylines	Lane-only; fast but specialised
Autodrive (this work)	Detection + lanes + steering	COCO-10, TuSimple, Local driving dataset	Boxes, lane masks, angle	Offline evaluation; single camera

Identified Gap

No single compact public system we are aware of combines explicit COCO-class detection, instance-level lane segmentation derived from sparse polylines, and a DAVE-2 regressor, while also documenting the data conversion and the metric caveats needed to interpret the results. Autodrive fills that gap as an engineering baseline. It does not claim a new network architecture; its novelty lies in the integrated design, the documented label-synthesis and sampling procedures, and the transparent evaluation.

III. MATERIALS AND METHODS

Research Framework

The workflow is: (1) acquire public datasets; (2) convert annotations into the formats each model expects; (3) verify labels visually; (4) train the three models independently; (5) evaluate each on its own split; (6) measure inference latency; and (7) assemble the outputs in the integration layer (Fig. 1). All training and evaluation ran on a Google Colab Tesla T4 GPU (14.9 GB usable memory) with Ultralytics 8.4.153 (lane) and 8.4.156 (detection), PyTorch 2.11.0+cu128 and Python 3.13. The steering network uses TensorFlow/Keras [28], [29] as listed in the project requirements, with OpenCV [30] for image handling.

Dataset Description

Object detection data (COCO-10). The detector is trained on ten categories from COCO 2017 [27]: person, bicycle, car, motorcycle, bus, truck, traffic light, stop sign, dog and cat. The first eight are directly relevant to driving; dog and cat represent animals on the road. Because 64,115 train2017 images contain a person but only 1,734 contain a stop sign, naive pooled sampling would reproduce COCO's extreme skew. We therefore use *round-robin* sampling: classes are visited in turn and one unseen image is taken per visit, so rare classes exhaust their pools before the budget is spent. This produced 8,000 training images, to which 80 background images (1%) were added to suppress false positives on empty scenes, giving 8,080. From val2017, 1,500 images were sampled and split into 975 validation and 525 test images (35% held out). Images whose only instances of a selected class were *iscrowd* regions were dropped, and 712, 78 and 45 crowd annotations were excluded from the train, validation and test labels. Table II(a) gives the resulting instance counts; the maximum-to-minimum training instance ratio is 24:1.

Table II(a). Dataset distribution: COCO-10 instances per class and split (images: 8,080 / 975 / 525).

Class	Train instances	Train share	Val instances	Test instances
person	24,661	43.8%	2,728	1,567
bicycle	2,887	5.1%	210	104
car	12,443	22.1%	1,276	642
motorcycle	3,054	5.4%	220	147
bus	2,188	3.9%	169	114
truck	2,914	5.2%	267	147
traffic light	4,768	8.5%	403	231
stop sign	1,118	2.0%	53	22
dog	1,221	2.2%	146	72
cat	1,013	1.8%	132	70
Total	**56,267**	**100%**	**5,604**	**3,116**

Lane data (TuSimple). TuSimple [11] provides highway driving clips recorded at 1280×720 resolution; each labelled clip contributes one annotated frame (the final frame, *20.jpg*). Annotations are JSON lines holding, for each frame, a list of lanes (each a vector of x-coordinates, with −2 marking "not visible") and a shared vector of fixed image rows (*h_samples*). The three training annotation files contained 3,626 valid records with no missing images. The dataset was obtained from the Kaggle mirror *manideep1108/tusimple* (≈21.6 GB archive).

Table II(b). Dataset distribution: TuSimple lane dataset as used in this work.

Item	Value
Annotated frames (valid records)	3,626 (0 missing images)
Resolution	1280×720 px (native); 640 px network input
Class	1 (lane)
Train / validation frames	2,900 / 726 (random split, seed 42)
Validation lane instances	2,732 (≈3.8 per frame)
Frames skipped during conversion	0 train, 0 val



Fig. 2. Label verification samples. Left: TuSimple lane ribbons overlaid on a validation frame (green fill, red outline). Right: COCO-10 bounding boxes drawn from the converted YOLO label files. In both cases the labels sit directly on the objects, confirming the coordinate conversions.



Fig. 3. Training-set class distribution of COCO-10 (log scale), showing the 24:1 spread between the most and least frequent classes.

Data Acquisition

All data are public. COCO images were downloaded individually via their *coco_url* fields using 32 parallel threads rather than fetching the full 19 GB archive. Every file was fully decoded with PIL after download; 8,080 train, 975 validation and 525 test images were verified with 0 failures. This check matters because a truncated JPEG has non-zero size and passes a naive existence test, then fails inside the data loader after several epochs. No personal or restricted data are involved; see Section XV for licensing notes.

Preprocessing

Detection labels. COCO boxes [x_{min}, y_{min}, w, h] in absolute pixels were clipped to the image, filtered (boxes with width or height ≤ 1 px were discarded; 0 were found) and converted to YOLO format (class, x_c, y_c, w, h, all normalised). COCO category ids (1–90 with gaps) were remapped to contiguous indices 0–9. This yielded 56,267, 5,604 and 3,116 boxes for train, validation and test.

Lane labels: centreline to polygon. TuSimple lanes are centrelines, but YOLO segmentation needs closed polygons. For each lane with at least three visible points $p_i = (x_i, y_i)$, consecutive points closer than 0.5 px are removed, unit tangents t_i are computed by central differences (one-sided at the ends), and unit normals n_i are obtained by a 90° rotation:

$$t_i = (p_{i+1} - p_{i-1}) / \|p_{i+1} - p_{i-1}\|, \quad n_i = (-t_{i,y}, t_{i,x}) \tag{1}$$

Lane markings appear wider nearer the camera, so the half-width grows linearly with image row, with $w_0 = 4$ px and $w_1 = 8$ px at native resolution and H = 720:

$$w(y) = w_0 + w_1 \cdot y / H \tag{2}$$

$$L_i = p_i + w(y_i)\, n_i, \quad R_i = p_i - w(y_i)\, n_i \tag{3}$$

The polygon is formed by walking up the left offset curve and back down the right one, then normalising coordinates by image width and height and clipping to [0, 1]:

$$\Pi = (L_1, \ldots, L_m, R_m, \ldots, R_1) \quad (4)$$

Labels are generated in a single pass; re-thickening existing polygons produces self-intersecting masks, so changing lane thickness requires editing w_0, w_1 and regenerating. Images were hard-linked rather than re-encoded, and every image was checked to have exactly one matching label file. Visual inspection (Fig. 2, left) confirmed that ribbons sit on the painted markings.

Dataset Splitting

COCO-10. Training images come from train2017. Validation and test images both come from val2017 and are disjoint: validation was used for early stopping and checkpoint selection, whereas the test split was untouched by any decision and supplies the headline numbers. COCO's train and val partitions are disjoint, so there is no image overlap. The pretrained initial weights were trained on all 80 COCO train2017 classes, so our procedure is a specialisation of an already COCO-trained model, not learning from scratch.

TuSimple. The 3,626 records were shuffled with seed 42 and split 80/20 into 2,900 training and 726 validation frames. Each record is a distinct clip, so adjacent frames of one video are not split across partitions. No separate test split was held out; lane results are therefore validation results and are optimistic to the extent that the checkpoint was selected on them (Section XI).

IV. PROPOSED METHOD

System/Model Architecture

Autodrive runs three networks on each camera frame (Fig. 1). The detector and lane network are YOLO11s variants of 9.42 M and 10.07 M parameters (21.4 and 32.9 GFLOPs after layer fusion). The steering network is a DAVE-2 CNN with 252,219 parameters (computed below), so it adds about 2.5% of the YOLO parameter count and a negligible compute cost. Table III lists the DAVE-2 layers and the two YOLO models.

Table III. Model architecture: DAVE-2 steering network (parameters computed from layer shapes) and the two YOLO11 models (fused, from training logs).

Layer	Parameters	Output
Input (YUV)	–	66×200×3
Normalisation	–	66×200×3
Conv1: 24 filters, 5×5, stride 2	1,824	31×98×24
Conv2: 36 filters, 5×5, stride 2	21,636	14×47×36
Conv3: 48 filters, 5×5, stride 2	43,248	5×22×48
Conv4: 64 filters, 3×3	27,712	3×20×64
Conv5: 64 filters, 3×3	36,928	1×18×64
Flatten	–	1,152
FC 100	115,300	100
FC 50	5,050	50
FC 10	510	10
Output (steering)	11	1
Total DAVE-2	**252,219**	
YOLO11s detector (10 classes)	9,416,670	100 layers, 21.4 GFLOPs
YOLO11s-seg lane network (1 class)	10,067,203	113 layers, 32.9 GFLOPs

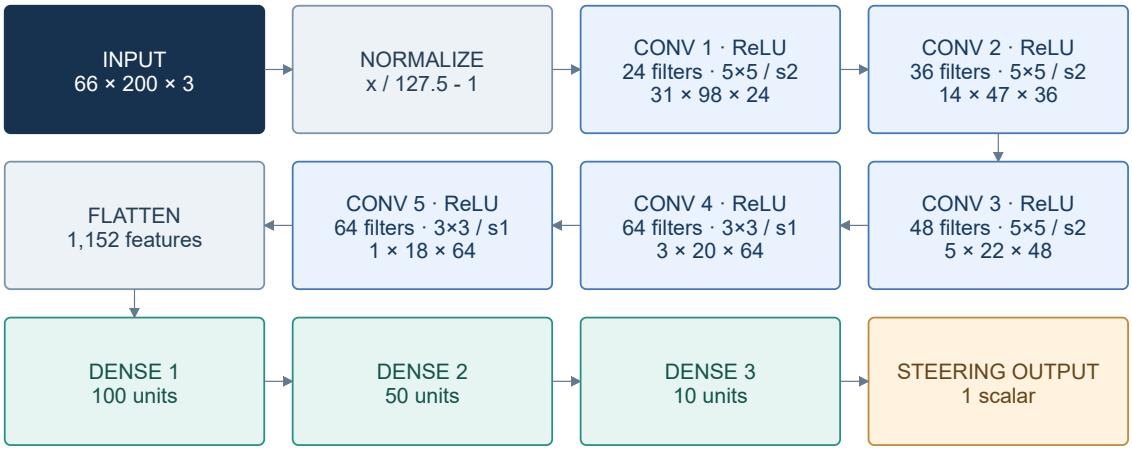


Fig. 4. DAVE-2 reference architecture [4]. Arrows show the forward pass; convolutional labels give filters, kernel/stride, and output dimensions (height x width x channels). Dense layers contain 100, 50, 10, and 1 unit.

Feature Extraction

DAVE-2. The input is a 66×200 RGB-to-YUV image. A fixed normalisation layer maps pixel values to [−1, 1] (x/127.5 − 1) so that it benefits from GPU execution and is not retrained. Three 5×5 stride-2 convolutions with 24, 36 and 48 filters capture coarse road structure, followed by two unstrided 3×3 convolutions with 64 filters. Output sizes follow the standard relation floor((n − k)/s) + 1 for an n×n input, k×k kernel and stride s. The original paper's figure labels the flatten size 1,164, whereas the shapes give 1×18×64 = 1,152; we use the computed value, which gives the ≈250 k parameters reported by NVIDIA [4].

YOLO11 backbones. Both YOLO11s networks use the same stem, a backbone built from C3k2 blocks, an SPPF pooling module and a C2PSA attention block, a neck that fuses three scales, and decoupled anchor-free heads [9], [8]. The segmentation model adds a prototype-mask branch in the style of YOLACT [18]. We rely on the vendor's published description of these blocks and did not modify the architectures.

Classification/Prediction

The detector outputs, per object, a box and one of ten class scores; the lane network outputs, per lane, a box, a confidence and a binary mask (class 0 = lane); the steering network outputs a single scalar steering command. NVIDIA expressed steering as the inverse turning radius to keep the target numerically stable near straight driving [4]; the unit convention used by the Autodrive steering data is radians internally; reported in degrees

Mathematical Formulation

YOLO11 training loss. The detection and segmentation losses are weighted sums of box, classification and distribution-focal terms (plus a mask term for segmentation), with the gains logged in our runs (box 7.5, cls 0.5, dfl 1.5):

$$L = \lambda_{box} L_{CIoU} + \lambda_{cls} L_{BCE} + \lambda_{dfl} L_{DFL} \; (+ L_{mask}) \tag{5}$$

Thin-structure mask IoU. To explain the metric behaviour in Section VIII, consider two parallel strips of equal width w whose centrelines are laterally offset by d < w. Their intersection has width w − d and their union w + d, so over a common length the IoU is

$$IoU(d) = (w - d) / (w + d) \tag{6}$$

At a 640-px network input (scale 0.5 from 1280×720) our ribbons are about 4 px wide at the horizon and 12 px at the bottom of the frame. A mere 2-px lateral error gives IoU = 0.71 at the bottom and 0.33 at the horizon, and a 1-px error already costs 0.6 at the horizon. High IoU thresholds (0.75–0.95) are therefore unreachable on distant lane segments even when the prediction is visually correct. Mean average precision averages AP over ten IoU thresholds:

$$mAP_{50\text{-}95} = (1/10) \sum_{\tau = 0.50,\, 0.55,\, \ldots,\, 0.95} AP_{\tau} \tag{7}$$

Steering loss. The DAVE-2 network is trained by minimising the mean squared error between the predicted and recorded steering command [4]:

$$L_{steer} = (1/N)\,\Sigma_{i=1\ldots N}\,(\theta_{pred,i} - \theta_{true,i})^2 \quad (8)$$

Classification metrics. Precision = TP/(TP + FP), recall = TP/(TP + FN) and F1 = 2PR/(P + R); a detection counts as a true positive when its IoU with an unmatched ground-truth object exceeds the threshold and its class is correct.

V. TRAINING AND OPTIMIZATION

Training Procedure

Each YOLO model is trained with the Ultralytics trainer: a forward pass produces box, class and (for segmentation) mask predictions; the loss of Eq. 5 is computed against task-aligned targets; gradients are back-propagated with automatic mixed precision; and weights are updated each iteration. After each epoch the model is evaluated on the validation split and the checkpoint with the best fitness is stored as *best.pt*. Early stopping triggers when validation fitness does not improve for *patience* epochs (15 for detection, 20 for lanes); neither run triggered it, so both completed their full schedule.

Detection run. Initialised from *yolo11s.pt* (head re-initialised for 10 classes), 50 epochs, batch 16, 640 px, cosine learning-rate decay, 3 warm-up epochs, mosaic augmentation with mosaic switched off for the final 10 epochs, seed 0 with deterministic mode, and disk caching of decoded images. It completed in 2.14 h at about 3.9 it/s on the T4.

Lane run. Initialised from *yolo11s-seg.pt*, 60 epochs, batch 16, 640 px, completing in 2.31 h. The notebook documents the reasoning behind each non-default setting: (i) *mosaic = 0* because stitching four frames creates lanes that terminate mid-air at seams, which is harmful for thin continuous structures; (ii) *mask_ratio = 2* because the default 4× mask down-sampling would shrink a 10-px lane to about 1 px and destroy the supervision signal; (iii) rotation, shear and perspective set to 0 because the camera is rigidly mounted and rotated road geometry is out-of-distribution; and (iv) horizontal flip kept at 0.5, since a mirrored road is plausible. The choice of the small model was made because with about 2.9 k training images larger variants overfit before they out-earn the small one.

Steering run. The local training script uses a sequential 80/20 train/validation split without shuffling, random initialization, 30 epochs, batch size 100, Adam at 1e-4, mean-squared error with L2 regularization (0.001), and keep probability 0.5. It configures no data augmentation or early stopping and does not select checkpoints using validation metrics.

Both YOLO runs used the Ultralytics *optimizer = auto* policy, which selects the optimiser and a size-dependent learning rate at run time from the schedule length; the resolved choice is recorded in the run logs. Weight decay was the default 5×10^{-4} and the initial and final learning-rate factors were $lr_0 = 0.01$ and $lr_f = 0.01$ in the logged configuration. Automatic mixed precision reduces memory and time on the T4. For the steering network, Adam [31] is the usual choice for DAVE-2 reimplementations; Adam, lr=1e-4

Hyperparameters

Table IV. Hyperparameters of the three modules (values taken from the logged training configurations).

Parameter	Detection (YOLO11s)	Lane (YOLO11s-seg)	Steering (DAVE-2)
Initial weights	yolo11s.pt (COCO)	yolo11s-seg.pt	random init
Epochs / patience	50 / 15	60 / 20	30 / none
Batch size	16	16	100
Input size	640×640	640×640	66×200
Optimiser	auto	auto	Adam
Loss	CIoU + BCE + DFL	CIoU + BCE + DFL + mask	MSE
LR schedule	cosine, 3 warm-up ep.	linear, 3 warm-up ep.	fixed 1e-4
Weight decay	5×10^{-4}	5×10^{-4}	L2 0.001
Mosaic	1.0 (off last 10 ep.)	0.0	–
Flip / scale / translate	0.5 / 0.5 / 0.1	0.5 / 0.4 / 0.1	none
HSV (h, s, v)	0.015, 0.7, 0.4	0.015, 0.7, 0.4	–
Rotation / shear / perspective	default	0 / 0 / 0	–
mask_ratio	–	2	–
Dropout	0	0	keep_prob 0.5
Seed / deterministic	0 / yes	0 / yes	not set
Mixed precision	yes	yes	no
Training time (T4)	2.14 h	2.31 h	not recorded

VI. EXPERIMENTAL DESIGN

Research Questions

1) **RQ1.** How accurately can a compact YOLO11s detector, fine-tuned on a class-balanced 8k-image COCO subset, recognise the ten road-relevant classes on held-out images?
2) **RQ2.** Can sparse TuSimple polylines be turned into instance-segmentation supervision, and what do the resulting mask and box metrics actually measure?
3) **RQ3.** What latency and memory budget does the combined perception stack require on a commodity GPU?

4) RQ4. The steering checkpoint is not accurate in this audit (MAE 22.96 deg, R2=-0.8769); lane consistency and closed-loop behavior were not evaluated.

Baseline Experiment

The experiments in this work are single-configuration training runs, not method comparisons, so there is no independent baseline detector. To give a reference point, we use each run's first-epoch validation snapshot, which reflects the pretrained backbone with only one epoch of adaptation: detection mAP50-95 of 0.331 and lane mask mAP50-95 of 0.173 (box 0.672). These are lower references rather than competing methods, and the improvement from them is reported in Section VIII.

Proposed-Method Experiment

Three experiments evaluate the proposed system. E1 (detection): train on 8,080 images, select on 975, and report on 525 held-out images with per-class metrics. E2 (lane): train on 2,900 frames, evaluate on 726 validation frames, and report box and mask metrics, a confusion matrix, and qualitative predictions. E3 (steering): compare predictions with labels on 505 validation frames sampled every 18 records; report MAE, RMSE, signed bias, R2, Pearson r, and percentage within +/-5 deg. No closed-loop evaluation was performed. Latency is measured using framework per-image preprocessing, inference, and postprocessing timers on the T4.

Ablation Study

No ablation experiments have been run yet. The contributions claimed here are an integrated system and documented data procedures, not a component-level architectural novelty, so ablations are not required to support them; but they would sharpen the conclusions. Table V lists the planned matrix and clearly marks each entry as not executed, so no result is implied.

Table V. Planned ablation matrix (not yet executed; no results are reported).

Experiment	Component changed	Question answered	Status
A1	Model scale n / s / m	Accuracy–latency trade-off	Not run
A2	Pretrained vs random init (detection)	Value of COCO pretraining on the 8k subset	Not run
A3	Mosaic on / off (lane)	Does mosaic harm thin structures?	Not run
A4	Ribbon half-width (w_0, w_1)	Sensitivity of mask mAP to label width	Not run
A5	mask_ratio 4 / 2 / 1	Mask resolution vs memory	Not run
A6	Input size 640 vs 960	Small-object recall (traffic light)	Not run
A7	Steering with / without side-camera augmentation	Recovery behaviour [4]	Not run

Comparative Experiment

No head-to-head comparison against other published detectors or lane methods was run. Published TuSimple results use the official accuracy/FP/FN protocol [11], which we did not compute, and our COCO-10 numbers use a different class set from published 80-class results, so direct comparison would be misleading. Adding the official TuSimple evaluation script is the most valuable next step for comparability.

VII. PERFORMANCE EVALUATION

Evaluation Metrics

Detection uses precision, recall, mAP50, mAP50-95, and mAP75 on the held-out test split. Lane segmentation reports box and mask metrics (Eq. 7). Latency is per-image preprocessing, inference, and postprocessing time. Steering is reported with MAE, RMSE, mean bias, R2, Pearson r, and within-+/-5-deg accuracy. No closed-loop autonomy score was measured; NVIDIA's intervention-based autonomy metric [4] is therefore not reported.

Confusion Matrix

Fig. 8 shows the confusion matrices. For lanes (Fig. 8a), at the confusion-matrix operating point (confidence 0.25), 2,706 of 2,732 ground-truth lanes are matched, with 119 predictions on background and 26 missed lanes, giving a recall of 0.990 (95% Wilson CI 0.986–0.993) and a precision of 0.958 (0.950–0.965). Intervals treat instances as independent, which is optimistic because lanes within a frame are correlated. For detection (Fig. 8b, normalised by true class), diagonal values range from 0.84 (cat) to 0.41 (truck).

Statistical Analysis

Each model was trained once with a fixed seed, so no mean ± standard deviation across seeds is available and differences of one or two points should not be over-interpreted. As a measure of sensitivity, the lane mask mAP50-95 was 0.363 when computed at the end of training and 0.389 on a standalone re-evaluation of the same checkpoint on the same frames, a spread of 0.026 attributable to evaluation settings; we report both. The COCO selection effect, the difference between validation and test mAP50-95, is +0.0144 (0.4855 vs 0.4711). Multi-seed repetition and bootstrap confidence intervals over images are listed as future work.

VIII. RESULTS

Training Results

Detection training (Fig. 5) improved smoothly: validation mAP50-95 rose from 0.331 after epoch 1 to 0.482 at epoch 50 (peak 0.484 at epoch 40) while the box, class and DFL losses fell to 1.012, 0.711 and 1.034. Validation precision and recall at the final epoch were 0.757 and 0.610. The plateau over the last ten epochs, which coincide with the mosaic-off phase, indicates convergence under this data budget; the flat curve rather than a decline shows no sign of overfitting.

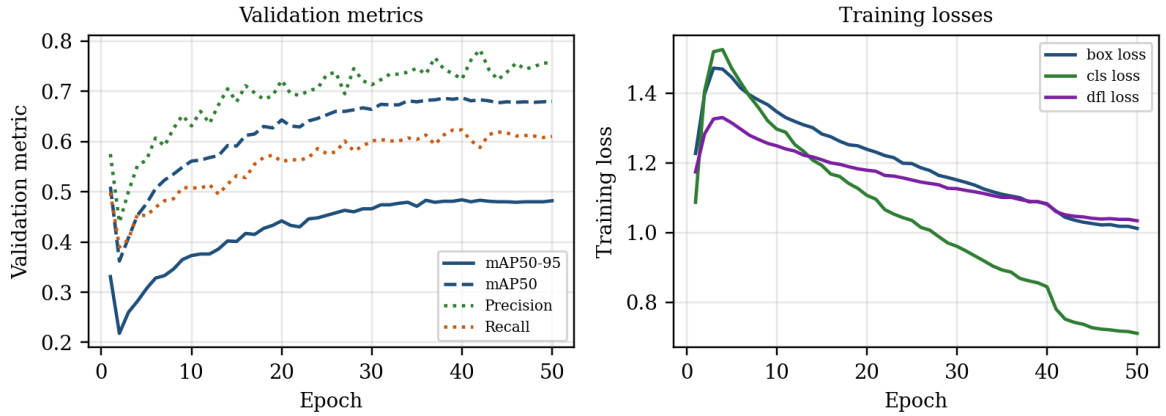


Fig. 5. Detection (COCO-10) training: validation metrics (left) and training losses (right) over 50 epochs.

Lane training (Fig. 6) converged more quickly in box terms: box mAP50-95 rose from 0.672 at epoch 1 to 0.831 at epoch 60 (peak 0.831 at epoch 56), and mask mAP50 reached 0.877. Mask mAP50-95 climbed from 0.173 to 0.363 and remained noticeably lower than the box value at every epoch. Epoch 2 shows a transient dip (mask mAP50-95 0.074) that recovers by epoch 3.

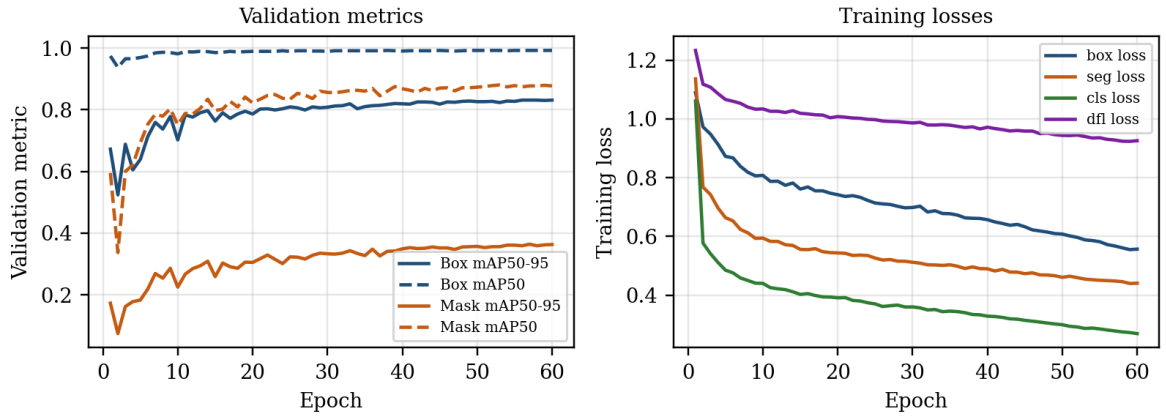


Fig. 6. Lane segmentation (TuSimple) training: validation metrics (left) and training losses (right) over 60 epochs.

Quantitative Results

Table VI(a). Main results: detection, COCO-10 (all classes, best.pt).

Split	Images	Instances	Precision	Recall	mAP50	mAP50-95	mAP75
Epoch-1 reference (val)	975	5,604	0.575	0.498	0.510	0.331	–
Validation	975	5,604	0.724	0.621	0.688	0.486	0.518
Test (held out)	525	3,116	**0.712**	**0.614**	**0.672**	**0.471**	**0.511**

On the held-out test split the detector reaches 0.471 mAP50-95 and 0.672 mAP50. The validation figure is 1.4 points higher, consistent with selection bias. Ultralytics reports roughly 47% mAP50-95 for YOLO11s on all 80 classes of COCO val2017; our number is of similar magnitude but is *not* comparable, because our class set, subset and splits differ.

Table VI(b). Main results: lane segmentation (TuSimple, 726 validation frames, 2,732 lanes). No held-out test split exists.

Evaluation	Box P	Box R	Box mAP50	Box mAP50-95	Mask P	Mask R	Mask mAP50	Mask mAP50-95
Epoch-1 reference	0.955	0.953	0.974	0.672	0.744	0.735	0.595	0.173
End of training (best.pt)	0.976	0.974	0.992	0.831	0.918	0.916	0.877	0.363
Standalone re-evaluation	0.976	0.974	0.992	0.831	0.925	0.923	0.890	0.389

Important: the lane mAP values are computed against the synthetic ribbons of Eqs. 1–4, not TuSimple's official point-wise accuracy and FP/FN metrics. They are a training-progress signal and must not be compared to published TuSimple leaderboard numbers.

Table VI(c). Steering checkpoint audit (n = 505); closed-loop autonomy was not evaluated.

Model	MAE (deg)	RMSE (deg)	Bias (deg)	R2	Pearson r	Within +/-5 deg
DAVE-2	22.9616	33.2350	+20.9977	-0.8769	-0.0404	9.11%

Class-Wise Results

Table VII. Class-wise detection results on the held-out test split (AP50-95 on validation shown for comparison).

Class	Test P	Test R	Test AP50	Test AP50-95	Val AP50-95	Test inst.
person	0.812	0.613	0.717	0.457	0.469	1,567
bicycle	0.686	0.399	0.498	0.270	0.311	104
car	0.763	0.607	0.680	0.456	0.406	642
motorcycle	0.802	0.633	0.771	0.472	0.422	147
bus	0.842	0.711	0.791	0.662	0.669	114
truck	0.561	0.463	0.521	0.331	0.347	147
traffic light	0.636	0.476	0.494	0.265	0.271	231
stop sign	0.488	0.607	0.619	0.556	0.736	22
dog	0.651	0.806	0.771	0.596	0.558	72
cat	0.882	0.829	0.861	0.645	0.667	70

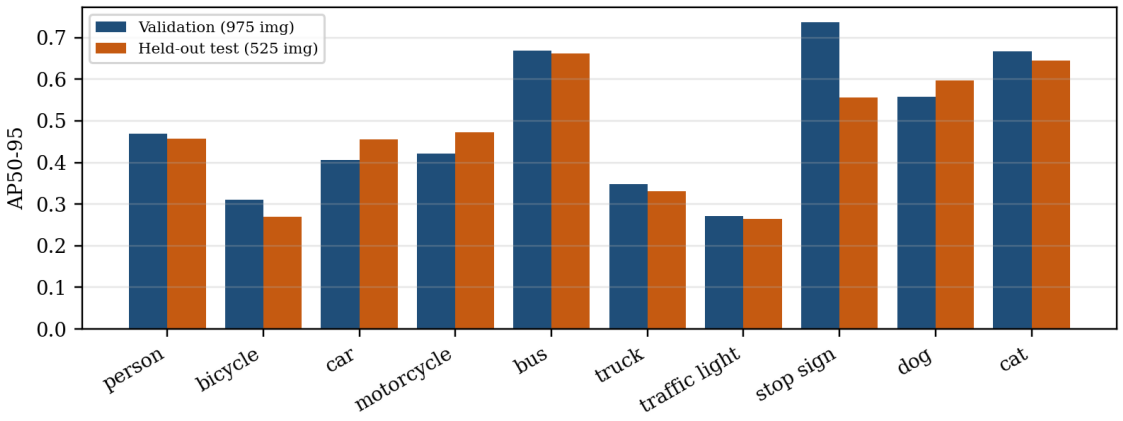


Fig. 7. Per-class AP50-95 on validation (975 images) and held-out test (525 images). Classes with few test instances (stop sign, 22) vary most between splits.

Bus (0.662), cat (0.645) and dog (0.596) are strongest, as are large, visually distinctive objects. Traffic light (0.265) and bicycle (0.270) are weakest, which is typical of small and thin objects at 640 px. The stop-sign drop from 0.736 (validation, 53 instances) to 0.556 (test, 22 instances) illustrates how unstable per-class AP becomes on small test counts. For the lane model there is a single class, so class-wise results are not applicable.

Confusion Matrix Results

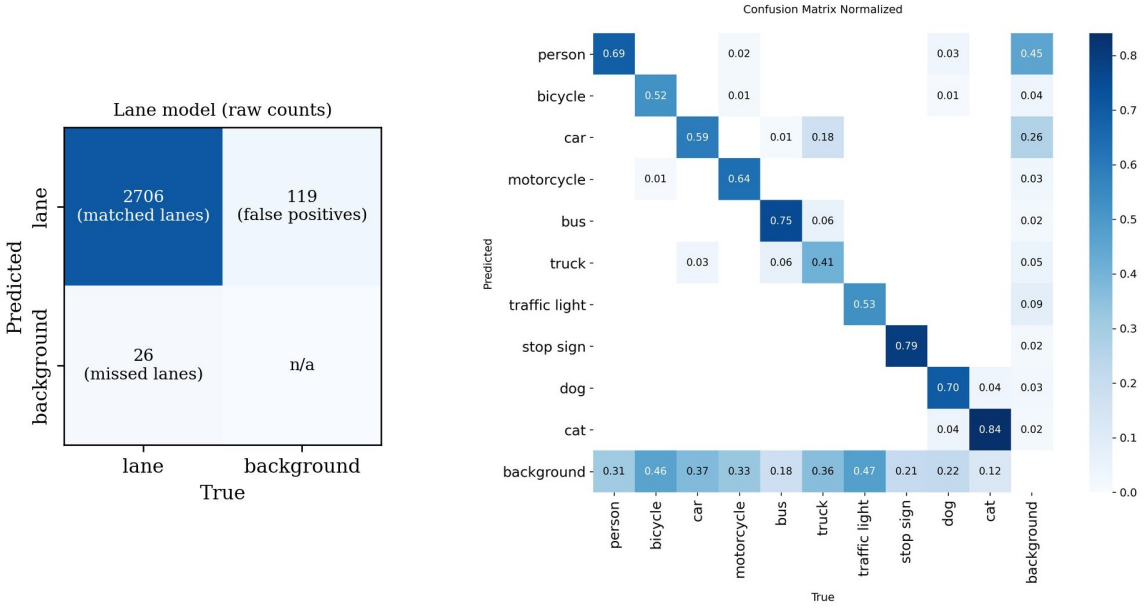


Fig. 8. Confusion matrices from the training runs (validation data). (a) Lane model, raw counts (columns: true class, rows: predicted). (b) Detection model, normalised by true class; the bottom row shows missed objects and the right column shows predictions on background.

The detection matrix shows that the dominant semantic confusion is **truck predicted as car** (0.18 of true trucks), with smaller truck/bus swaps (0.06 each way), which matches the ambiguity of COCO's own vehicle annotations. More mass sits in the "missed" row: 31% of persons, 46% of bicycles, 47% of traffic lights and 37% of cars are not detected at the evaluated confidence threshold, which accounts for the recall of about 0.62. The right-hand column shows that, of the predictions made on background regions, 45% are persons and 26% are cars, so false positives concentrate in the most frequent classes.

Qualitative Results

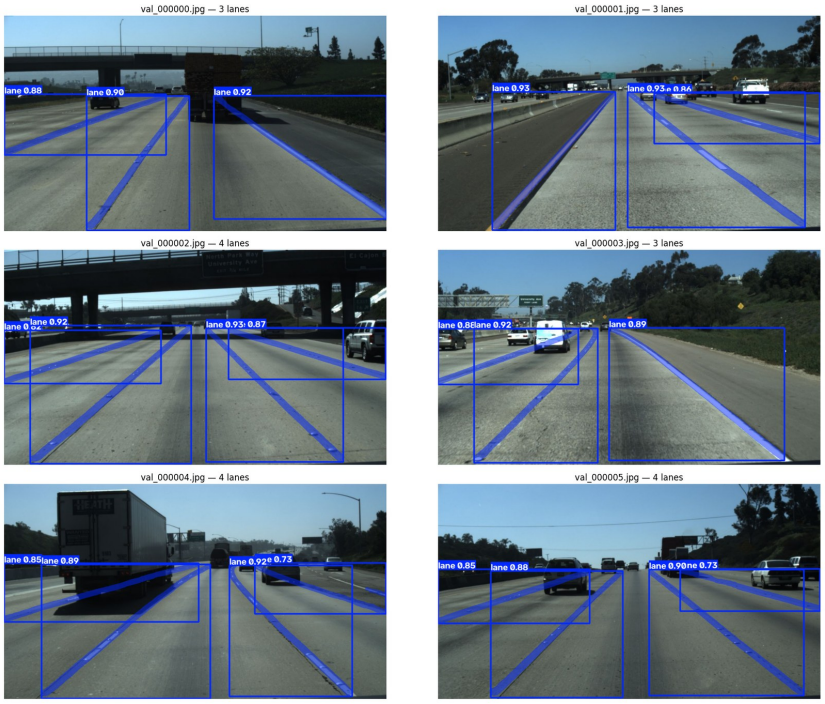


Fig. 9. Lane predictions on six validation frames (conf ≥ 0.25). Three or four lanes are found per frame with confidences of roughly 0.73–0.93, and the masks follow the painted markings, including under overpass shadows and behind a truck. Large overlapping boxes are an artefact of diagonal, thin lanes.



Fig. 10. Detection predictions on six held-out test images (conf ≥ 0.25, NMS IoU 0.7), e.g. a cat at 0.91 confidence and buses at 0.94 and 0.86 with several partially occluded persons.

Latency and Footprint

Table VIII. Per-image latency on a Tesla T4 (milliseconds, from framework timers) and weight size.

Module	Source of timing	Pre	Inference	Post	Total	Weights
Detection	Test evaluation (batch 16)	1.9	7.3	1.9	11.1	19.2 MB
Lane segmentation	Re-evaluation (batch 16)	1.2	8.1	2.5	11.8	20.5 MB
Detection	Single-image predict	3.7	11.0	2.3	17.0	–
Lane segmentation	Single-image predict	1.1	10.2	3.0	14.3	–
Both, sequential	Batch-16 sum / single-image sum				**22.9 / 31.3**	39.7 MB

Summing per-module totals gives 22.9 ms (batch-16 timers) to 31.3 ms (single-image timers), or about **32–44 frames per second** for the two YOLO modules run sequentially on one T4. This is a *projection* from separate benchmarks, not an end-to-end video measurement: it excludes frame capture, the steering CNN (a few hundred thousand parameters, expected to be small), display and control I/O. The in-training validator reported a slower 14.5 ms lane inference, a reminder that timer settings affect reported latency.

IX. DISCUSSION

Major Findings

Detection is serviceable but recall-limited. With only 8,080 images the specialised detector reaches 0.471 test mAP50-95, strong on large classes and weak on small ones. Precision (0.712) exceeds recall (0.614), which means that lowering the confidence threshold can recover objects at the price of more false alarms; for a safety-oriented system recall of small, vulnerable road users such as bicycles and persons is the figure that matters, and it is the weakest part of the current result.

The lane network finds lanes reliably; the mask metric is the wrong lens. Box mAP50 of 0.992 and a confusion-matrix recall of 0.990 say that lanes are almost always found, but mask mAP50-95 is only 0.36–0.39. Eq. 6 explains why: with ribbons 4–12 px wide at network scale, boundary offsets of only a pixel or two destroy IoU at strict thresholds. Furthermore, box mAP50-95 (0.83) *overstates* localisation for the opposite reason: a diagonal thin lane has a huge bounding box that is easy to overlap (Fig. 9). Neither number is a good measure of what steering needs, which is the lateral position of the lane centre at a few look-ahead rows. We therefore recommend adding TuSimple's official point-wise accuracy and FP/FN metric, and a centreline-offset error in pixels.

Comparison With Previous Work

Direct numerical comparison with prior work is not possible, for the reasons set out in Section VI: our lane metric is non-standard, our detection class set is a subset, and the steering protocol differs from NVIDIA's closed-loop road tests [4]. Qualitatively, the design differs from LaneNet and SCNN [22], [21] by using a general-purpose instance-segmentation head rather than a purpose-built lane network, trading some specialisation for a shared toolchain with the detector; and it differs from DAVE-2 alone by running explicit perception alongside the regressor.

Ablation/Error Analysis

Without ablations we cannot attribute performance to individual design choices, and the rationale for settings such as *mosaic = 0* and *mask_ratio = 2* rests on reasoning, not measured effects (Table V). The failure modes we can document from the data are: truck/car confusion (COCO ambiguity), missed small objects (low resolution at 640 px), unstable per-class AP on small test counts, and thin-structure IoU sensitivity.

Engineering Significance

The two YOLO models total roughly 40 MB and, by the projection above, can process a video stream at tens of frames per second on a T4, and are small enough to be candidates for embedded GPUs after export to formats such as ONNX or TensorRT (not benchmarked here). The DAVE-2 network is orders of magnitude smaller than either YOLO model. The practical value is a reproducible baseline whose two YOLO modules train in about four and a half GPU-hours in total; the practical caution is that nothing here is validated for on-road use.

X. EXPLAINABILITY / INTERPRETABILITY

Interpretability Method

Explainability analysis for the steering network is **not yet performed**, and we therefore present no saliency figure. The intended method is VisualBackProp [10], which NVIDIA used to show that DAVE-2 relies on lane markings and road edges, and Grad-CAM [32] as a complementary gradient-based map. Both operate on the trained steering CNN and therefore require its weights; they should be run on frames spanning straight roads, curves and occlusions.

Interpretation

What can be said now is that the lane-segmentation branch provides an *interpretable intermediate* signal: its masks visibly follow the painted markings (Fig. 9). A natural consistency test, left for future work, is to check whether the steering prediction correlates with the lateral offset and heading of the segmented lane centre, and whether the VisualBackProp saliency overlaps the segmented lane pixels. Divergence would flag frames on which the regressor is unreliable.

XI. LIMITATIONS AND FUTURE WORK

Limitations

- **No closed-loop evaluation.** Offline accuracy is a weak predictor of driving quality [14]; closed-loop trials in a simulator [26] or on a test track are required.
- **Non-standard lane metric.** Mask mAP is computed against synthetic ribbons whose width (w_0, w_1) is a free choice, and the official TuSimple metric was not computed.
- **No held-out lane test split and a possibly optimistic split.** The 726 validation frames were also used to pick the checkpoint. TuSimple clips come from a limited set of US highway recording sessions in good-to-medium weather, so a random clip-level split can leave very similar scenes in both partitions.
- **Small, shifted detection data.** COCO is not a driving dataset; 8,080 training images and a test split with only 22 stop signs and 72 dogs yield noisy per-class AP. Initial weights already saw all COCO train2017 classes.
- **Single runs.** One seed per experiment, no confidence intervals across runs, and no ablations.
- **Latency is a projection.** End-to-end throughput was not measured, no export or quantisation was tested, and the T4 is not an automotive-grade device.
- **Single camera, no temporal modelling.** No tracking, no multi-frame context, and covariate shift remains an open issue for behaviour cloning [15].

Future Work

- Add the official TuSimple evaluation, centreline-offset error, and CULane or BDD100K [33] for broader coverage.
- Train the detector on driving-specific datasets such as KITTI [34], BDD100K [33] and nuScenes [35].
- Run the ablation matrix of Table V with multiple seeds and bootstrap confidence intervals.
- Close the loop in CARLA [26], report the NVIDIA autonomy metric and add DAgger-style data aggregation [15].
- Export to ONNX/TensorRT with INT8 quantisation, re-validate accuracy after export, and benchmark on embedded GPUs.
- Fuse the three branches: use lane geometry and detected obstacles to gate or sanity-check the steering output, and add multi-object tracking.

XII. CONCLUSION

Autodrive combines a YOLO11s road-object detector, a YOLO11s-seg lane network built from converted TuSimple polylines, and a DAVE-2 steering regressor in a modular perception-to-steering prototype. On 525 held-out COCO-10 images, detection reaches 0.672 mAP50 and 0.471 mAP50-95. Lane validation reports 0.992 box mAP50 and 0.877-0.890 mask mAP50; thin-mask geometry limits mask mAP50-95 to 0.363-0.389. Component timings project to 32-44 FPS for the two YOLO modules. In a systematic sample of 505 steering-validation frames, MAE was 22.9616 deg, RMSE 33.2350 deg, mean bias +20.9977 deg, R2=-0.8769, Pearson r=-0.0404, and 9.11% fell within +/-5 deg. These steering results do not establish reliable control, and closed-loop performance was not measured. The paper documents its data conversion, model pipeline, results, and limits; road use requires independent validation, official lane metrics, driving-domain coverage, and closed-loop safety evaluation.

XIII. DATA AVAILABILITY

COCO 2017 images and annotations are public via cocodataset.org [27]. TuSimple [11] was obtained through the Kaggle mirror manideep1108/tusimple; converted YOLO-format datasets are regenerated by the notebooks and are not redistributed. Steering frames came from a local driving dataset; the original collection source and redistribution licence were not documented.

XIV. CODE AVAILABILITY

Project code is at https://github.com/HA2345567/Autodrive [36] (folders: data, model_training, saved_models, src, tests). The detection and lane pipelines are provided as Colab notebooks (*coco_object_detection_yolo11* and *tusimple_lane_yolo11*). Software: Python 3.13, PyTorch 2.11.0+cu128, Ultralytics 8.4.153/8.4.156, TensorFlow/Keras, OpenCV, NumPy, pandas, Matplotlib. Hardware: Tesla T4 (Colab). The YOLO11 code and weights are distributed by Ultralytics under the AGPL-3.0 licence unless an enterprise licence is obtained, which affects redistribution of derived systems.

XV. ETHICAL CONSIDERATIONS

No human-subject data were collected. COCO and TuSimple are public research datasets used under their respective terms; COCO images carry varied Flickr licences, and the datasets contain incidental images of people and vehicles. **Safety:** Autodrive is a research prototype. It has not been validated in closed loop or on public roads and must not be used to control a vehicle outside controlled research settings.

XVI. DECLARATIONS

Funding: No external funding.

Conflict of Interest: None declared.

Author Contributions: All authors contributed to the project and manuscript; individual roles were not documented.

REFERENCES

References are numbered in order of first citation (IEEE style).

[1] SAE International, “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” SAE Standard J3016_202104, 2021.

[2] D. A. Pomerleau, “ALVINN: An autonomous land vehicle in a neural network,” in *Advances in Neural Information Processing Systems 1*, 1989, pp. 305–313.

[3] Y. LeCun, U. Muller, J. Ben, E. Cosatto, and B. Flepp, “Off-road obstacle avoidance through end-to-end learning,” in *Advances in Neural Information Processing Systems 18*, 2005.

[4] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, “End to end learning for self-driving cars,” arXiv:1604.07316, 2016.

[5] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024 (arXiv:2306.16927).

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[7] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023.

[8] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” arXiv:2410.17725, 2024.

[10] M. Bojarski, P. Yeres, A. Choromanska, K. Choromanski, B. Firner, L. Jackel, and U. Muller, “Explaining how a deep neural network trained with end-to-end learning steers a car,” arXiv:1704.07911, 2017.

[11] TuSimple, “TuSimple lane detection benchmark,” 2017. [Online]. Available: https://github.com/TuSimple/tusimple-benchmark

[12] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, “End-to-end driving via conditional imitation learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 4693–4700.

[13] H. Xu, Y. Gao, F. Yu, and T. Darrell, “End-to-end learning of driving models from large-scale video datasets,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.

[14] F. Codevilla, E. Santana, A. M. López, and A. Gaidon, “Exploring the limitations of behavior cloning for autonomous driving,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9329–9338.

[15] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. Artif. Intell. Stat. (AISTATS)*, 2011, pp. 627–635.

[16] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017.

[17] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2961–2969.

[18] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “YOLACT: Real-time instance segmentation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9157–9166.

[19] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12993–13000.

[20] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, “Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,” in *Advances in Neural Information Processing Systems 33*, 2020.

[21] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, “Spatial as deep: Spatial CNN for traffic scene understanding,” in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 7276–7283.

[22] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans, and L. Van Gool, “Towards end-to-end lane detection: An instance segmentation approach,” in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2018, pp. 286–291.

[23] Y. Hou, Z. Ma, C. Liu, and C. C. Loy, “Learning lightweight lane detection CNNs by self attention distillation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1013–1021.

[24] Z. Qin, H. Wang, and X. Li, “Ultra fast structure-aware deep lane detection,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 276–291.

[25] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. De Souza, and T. Oliveira-Santos, “Keep your eyes on the lane: Real-time attention-guided lane detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 294–302.

[26] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. Conf. Robot Learn. (CoRL)*, 2017, pp. 1–16.

[27] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[28] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: https://www.tensorflow.org

[29] F. Chollet et al., “Keras,” 2015. [Online]. Available: https://keras.io

[30] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

[31] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[32] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[33] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, “BDD100K: A diverse driving dataset for heterogeneous multitask learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2636–2645.

[34] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354–3361.

[35] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11621–11631.

[36] Autodrive source code repository. [Online]. Available: https://github.com/HA2345567/Autodrive